

5. Vestavěné systémy (mikrokontrolér, periférie, rozhraní, převodníky)

5. Vestavěné systémy

SZZ okruh 5 (FIT BUT). **Vestavěný systém** = jednoúčelový počítač zabudovaný do zařízení, které řídí. Cíle: autonomie, reaktivnost (reálný čas), kritičnost.

Shrnutí

Mikrokontrolér (MCU)

Integrovaný obvod = **kompletní počítač** (CPU, RAM, paměť programu, oscilátor/krystal, GPIO), často + AD/DA převodník, časovače, **watchdog** (reset při zaseknutí).

- **Von Neumann** — společná paměť pro data i program, flexibilní (i samomodifikující kód), ale sběrnice je úzké hrdlo → pomalejší.
- **Harvard** — oddělené paměti programu a dat, v jednom taktu lze načíst instrukci i operand → rychlejší.

[[media/szz-05/media/image1.png]] *Von Neumannova vs. Harvardská architektura.*

Instrukční sada [[okruhy/07-princip-cinnosti-pocitace-pipelining | RISC × CISC]].

Sériová rozhraní

- **UART** — asynchronní; rámec start bit → data (LSB→MSB) → paritní → stop bit(y); smluvený **baud rate**.
- **SPI** — synchronní, full-duplex, Master-Slave; vodiče MISO, MOSI, SPCK, SS (výběr Slave, aktivní v log. 0).
- **I2C** — synchronní, half-duplex; dva vodiče SDA + SCL (pull-up); 7bitová adresa → až 128 zařízení; rámce s ACK; start/stop condition.

[[media/szz-05/media/image7.png]] *SPI — Master-Slave, vodiče MISO/MOSI/SPCK/SS.*

Převodníky

- **ADC** (analogově-číslicový): **Flash ADC** (kombinační, $2^N - 1$ [[okruhy/02-kombinacni-logicke-obvody|komparátorů]] + prioritní kodér, rychlý, drahý); **aproximační ADC** (sekvenční, SAR, půlení intervalů, $O(N)$).
- **DAC** (číslicově-analogový): obvykle **R-2R odporový žebřík**.

[[media/szz-05/media/image9.png]] *Flash ADC — napěťový žebřík odporů, soustava komparátorů a prioritní kodér.*

Periferie a řízení

Displej (7segment), klávesnice (dělič napětí + ADC, nebo kodér), motory/LED/bzučák přes **PWM** (řízení střídý) nebo DAC. Připojování a obsluhu řeší podrobně [[okruhy/06-periferie-preruseni-dma-sberrnice]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Mikrokontrolér □ [[#Mikrokontrolér (MCU)]] - Co je MCU, z čeho se skládá? □ Kompletní počítač na čipu: CPU, RAM, paměť programu, oscilátor, GPIO + periferie (časovače, čítače, watchdog, ADC/DAC). - Watchdog? □ Resetuje MCU, pokud nedostane pravidelný signál □ ochrana proti zaseknutí. - Von Neumann vs. Harvard? □ VN: společná paměť dat i programu (úzké hrdlo sběrnice); Harvard: oddělené paměti □ instrukce i operand v jednom taktu.

Rozhraní / komunikace □ [[#Sériová rozhraní]] - UART vs. SPI vs. I2C? □ UART asynchronní (start/stop bit, baud rate); SPI synchronní full-duplex master-slave (MOSI/MISO/SCLK/SS); I2C synchronní half-duplex, 2 vodiče (SDA/SCL), 7bitová adresa. - Synchronní vs. asynchronní přenos? □ Synchronní přenáší i hodiny; asynchronní si je generuje příjemce ze smlouveného baud rate.

Převodníky (ADC/DAC) □ [[#Převodníky]] - Flash vs. aproximační ADC? □ Flash = paralelní pole komparátorů (rychlý, drahý); aproximační = SAR, pulzení intervalů, $O(N)$. - Proč schodovitý převodní graf? □ Digitální výstup je diskretní (kvantovaný), ne spojitá čára; přesnost roste s počtem bitů. - Role S/H obvodu? □ Sample & hold navzorkuje a drží napětí po dobu převodu.

Plné znění (ke studiu)

Evropské strategické iniciativy definuje embedded systems jako kombinaci hardwaru a softwaru, jejímž smyslem je řídit externí proces, zařízení nebo systém.

Vestavěný (Vestavný/Embedded) systém je jednoúčelový počítač, který je **zabudován** do zařízení, které ovládá. Na rozdíl od univerzálních počítačů jsou vestavěné systémy **specializované** (nikoliv součástkami) a mají **předem určenou činnost**, kterou řídí. Např. výdej nápojů v automatu, volba pracovního režimu a samotná činnost praní v pračce, automobil tesla atd. Snaží se o:

- **Autonomie** - Funkčnost bez lidského zásahu.
- **Reaktivnost** - Odezva na podnět v reálném čase.
- **Kritičnost** - Vliv odchylek na normální chování na bezpečné plnění úlohy.

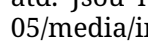
Vlastnosti vestavných systémů:

- Omezená množina aplikačních systémů (vykonává pouze jeden úkol).
 - Často pouze jeden program na celý život.
- Často zpracovává fyzikálních veličin.
- Měly by být spolehlivé, bezpečné, zabezpečené a efektivní.
- Hlavní interakce nemusí být s člověkem.
- Ideálně by člověk neměl ani přemýšlet nad tím, že pracuje s počítačem.

I když je většinou funkce vestavěného systému **specializovaná**, nechceme pro každý systém vyvíjet **nový** integrovaný obvod např. s automatem, který by popisoval jeho činnost. Bylo by to časově a hlavně **finančně** náročné, navíc by poté nešlo provádět případné **změny** (i když v některých extrémních případech ano - optimalizace). Proto je dnes základem téměř každého vestavěného systému

mikrokontrolér a na něj jsou napojeny dle potřeby periferní zařízení (senzory, display, reproduktor, ...), které jsou opět vyráběny ve velkých počtech. Samozřejmě existuje nespočet mikrokontrolérů, které se liší především cenou, výkonem, spotřebou, pamětí, a je na návrháři vestavěného systému, aby vybral pro účel navrhovaného systému vhodný. Stejně tak to platí o různá periferní zařízení.

Mikrokontrolér (MCU)

Integrovaný obvod implementující **kompletní počítač** (ALU, paměť, síťové rozhraní, ..), který lze naprogramovat např. v jazyce C (nemusí být moc výkonný). Navíc od běžných procesorů často mikrokontroléry obsahují např. AD převodník, DA převodník, programovatelné časovače atd. Jsou hlavní komponentou vestavných systémů. Dle architektury je dělíme na: 

Von Neumannova architektura MCU

- **Společná paměť pro data i program,**
- **flexibilnější** pro měnící se aplikace,
- umožňuje **samomodifikující** se kód,
- instrukce (program) i operandy (data) se musí z paměti vybírat **postupně**, sběrnice je úzkým hrdlem. Je **pomalejší** než Harvardská.
- Jednodušší implementace, není třeba rozlišovat čtení dat od čtení programu.

Harvardská architektura MCU

- **Paměť programu a dat jsou odděleny** (dvojí adresový prostor a dvě paměti),
- kód může být uložen v paměti jiného typu (FLASH) než data (RAM),
- Umožňuje použít **rozdílnou velikost buňky** paměti – např. 8 bitů (1 byte) pro data a 18 bitů pro instrukci (podle délky instrukce, tak aby jedno čtení načetlo vždy celou instrukci),
- Umožňuje v jednom taktu získat instrukci a **zároveň** její operandy - rychlejší než Von Neumann.



Harvard Architecture versus Von Neumann Architecture

Instrukční sada CISC (Complex Instruction Set Computing)

- **Složitá instrukční sada.**
- Mnoho typů instrukcí s mnoha variantami a mnoha adresovacími režimy (jedna instrukce může kombinovat více elementárních operací)
- Typicky memory-to-memory instrukce (výběr operandů a uložení výsledků je součástí instrukce).
- instrukce nemusí mít stejnou délku - náročnější na dekodování
- provedení instrukce trvá více taktů
- nepoužívá mnoho registrů, někdy pouze **střadač**.
- Na čipu dominuje logika pro implementaci instrukcí.
- procesory Intel.

Instrukční sada RISC (Reduced Instruction Set Computer)

- **Redukovaná instrukční sada.**
- Oproti CISC malé množství instrukcí, které provádí pouze **elementární** operace, časté instrukce jsou LOAD a STORE
- všechny instrukce jsou stejně dlouhé (stejný počet bitů v paměti) a typicky trvají **jeden** takt
- využívá velké množství registrů
- na čipu dominují registry (paměť) oproti obvodům pro vykonávání instrukcí
- procesory s touto architekturou mají menší spotřebu, dnes začínají převládat
- Apple Silicon

RISC versus CISC

Struktura MCU

- **Procesor**
- **Operační paměť** - RAM
- **Paměť programu** - **EEPROM**, PROM, ROM, **FLASH**, ...
- **Oscilátor** - Zdroj hodinového signálu (RC/**Krystal**)
- **Vstupně výstupní rozhraní (GPIO - general purpose input output)** - AD/DA převodník, paralelní/sériové porty, Ethernet... Obvykle jeden port lze využít k více účelům, aby nemělo MCU zbytečně velký počet výstupů.

Periferie MCU

- **Časovač** - Měření času, např. RTC (real time clock), **TODO**
- **Hodiny** - přesnější zdroj hodinového signálů (**krystal**)
- **Čítač** - Viz otázka 3
- **Watchdog** - Stará se o to aby nedošlo k “zaseknutí” MCU při chybě. Pokud mu nepřijde pravidelné upozornění, tak resetuje celý MCU.
- **FPGA** - Programovatelné hradlové pole
- **Řadič přerušení** - klávesnice
- **Senzory, displej, reproduktor, signalizační LED, ...**

Sériové rozhraní

Slouží pro komunikaci s MCU. Komunikace probíhá po jednotlivých bitech (**sekvenčně**). Bity jsou přenášeny po jediném vodiči jeden za druhým. Je třeba jednoznačně určit, v kterém okamžiku je na datovém vodiči hodnota kterého bitu.

- **Synchronní přenos:** Spolu s daty je přenášén i hodinový signál, který určuje, kdy lze číst další bit. (nutné jsou minimálně 2 vodiče)
- **Asynchronní přenos:** hodinový signál není přenášén, přijímač i vysílač si jej generují sami (musí mít smluvený **baud rate**). Je nutné zajistit dostatečnou přesnost hodinového signálu a jeho synchronizaci.
 - baud rate (Bd)

- * počet symbolů (bitů, bytů, případně jiných přenášených prvků) přenesených za sekundu
- * u sériové komunikace například 1 baud = 1 bit/s

UART (Universal asynchronous receiver-transmitter)

Zařízení umožňující asynchronní komunikaci. Synchronizace je řešena pomocí **start bitu**, který je na začátku každého přenášeného rámce. Jedná se o přechod z klidové úrovně **log. 1** do **log. 0**. Příjímá podle něj **synchronizuje** hodiny, **za** tímto bitem již následují bity datové (8 bitů). Po datových bitech může následovat **paritní bit** a **jeden** nebo **dva stop bity** (vrátí linku opět do klidové úrovně **log. 1**). Přenos je tedy **start bit - LSB - ... - MSB - (paritní bit) - stop bit - (stop bit)**. Příklad na obrázku přenáší hodnotu **0x3B = 0b00111011**.

[[media/szz-05/media/image8.png]]

Komunikace u UART je **většinou plně duplexní (full-duplex)**, může být ale i **half-duplex** nebo pouze **simplex** (pokud přijímající stanice neobsahuje vysílač).

[[media/szz-05/media/image4.png]]

SPI (Serial Peripheral Interface)

Jedná se **synchronní sériové** rozhraní typu **Master-Slave**, které je **plně duplexní**. V každém okamžiku **vždy** probíhá přenos **oběma** směry. Levná varianta pro připojování periferních zařízení, lze pomocí SPI realizovat ale i sběrnici.

- **Master:** na SPI sběrnici je vždy **jediný**. Je **zdrojem** hodinového signálu, **iniciuje** a **řídí** komunikaci. Obvykle se jedná o **MCU**.
- **Slave:** ke sběrnici může být připojeno **více** zařízení typu slave, **Master určuje** se kterým v daný okamžik komunikuje.

Vodiče, po kterých probíhá přenos, nebo jej řídí, jsou následující:

- **MISO** (Master In Slave Out) - vodič, po kterém jsou přenášena data od **Slave (zapisuje)** k **Master (čte)**.
- **MOSI** (Master Out Slave In) - vodič, po kterém jsou přenášena data of **Master (zapisuje)** k **Slave (čte)**
- **SPSCK** (SPI Serial Clock) - vodič po kterém je přenášén hodinový signál. Na zařízení Master je to vždy **výstup** a na zařízení Slave **vstup**.
- **SS** (Slave Select) - vodič, který vybírá Slave pro komunikaci. Na zařízení Master je **výstupem**. V **log. 1** je neaktivní (stejně jako UART), v **log. 0** je **aktivní**. V log. 0 musí být po **celou** dobu přenosu. V daném okamžiku může **Master** takto aktivovat **maximálně jeden** modul **Slave**, se kterým chce komunikovat. Zařízení, které mají SS v **log. 1 nezasahují** do probíhající komunikace.

[[media/szz-05/media/image7.png]]

I2C

Jedná se o **synchronní sériové** rozhraní, které funguje na principu **Master-Slave** komunikující způsobem **half-duplex**. Rozhraní tvoří dva vodiče: **SDA (Serial Data** - vodič po kterém jsou přenášena data) a **SCL (Serial Clock** - vodič nesoucí synchronizační signál - hodiny). Oba vodiče jsou zapojeny přes **pull up** rezistory k **Vdd** (log. 1), to znamená v klidu jsou oba vodiče v **log. 1**. Přenos dat probíhá následovně:

- **Start condition** (zahájení komunikace): komunikaci vždy zahajuje Master (obvykle MCU) při **SCL v log. 1** změnou úrovně **SDA z log. 1 na log. 0 (1->0)**.
- **Komunikace**: Data jsou **vzorkována** z vodiče **SDA s náběžnou** hranou na vodiči **SCL**, změna úrovně na vodiči **SDA** musí být tedy prováděna, když je **SCL v log. 0**.
- **Stop condition** (ukončení komunikace): Master na vodiči **SDA** generuje hranu z **log. 0 do log. 1 (0->1)**. **SCL** musí být v **log. 1**, jinak by šlo pouze o změnu dat.

Standardně se v datovém rámci přenáší **1 byte** užitečných dat (**po bitech**), každý rámeček je zakončen s **ACK** (potvrzení Slave, že data obdržel). Volbu komunikujícího zařízení (Slave) Master realizuje pomocí **adresy** na začátku komunikace v 1. **adresovém** rámci. Tato adresa má **7 bitů** (na **I2C sběrnici** tak může být připojeno až **128** zařízení Slave - Master je vždy **jeden**), **osmý** bit určuje, jestli půjde o **čtení** nebo **zápis**.

[[media/szz-05/media/image12.png]]

[[media/szz-05/media/image2.png]]

[[media/szz-05/media/image11.png]]

Paralelní rozhraní

Slouží pro komunikaci komponentů MCU. Je současně posíláno více bitů po více vodičích. Tento způsob může být rychlejší, ale musí se brát ohled na interference paralelních vodičů (magnetická indukce). <https://electronics.stackexchange.com/questions/21675/what-should-i-know-about-interference-between-wires-in-a-multi-conductor-cable>

Převodníky

Umožňují komunikaci se světem, který je analogový. Slouží většinou k získávání informací z různých senzorů, které převádí měřené veličiny na napětí a toto napětí se poté převádí pomocí **AD** převodníku na binární hodnotu, kterou dokáže interpretovat MCU. U **DA** převodníku pak MCU pomocí binární hodnoty může vysílat potřebné napětí na výstupu, který např. nastavuje teplotou.

AD převodník (analog-to-digital converter)

Převádí **analogový** (spojitý) signál na **digitální** (diskrétní - **binární číslo**, které dokáže interpretovat MCU). Naměřené hodnoty MCU ale dostává v rozsahu 0 až $2^N - 1$ a musí je převést na **odpovídající** hodnotu napětí. Existuje více druhů AD převodníků, dva základní jsou:

- **Flash ADC** (direct-conversion ADC): Jedná se pouze o **kombinační** obvod, převod probíhá **paralelně** s **konstantní** rychlostí. N bitový převodník je tvořen $2^N - 1$ **komparátory**, které jsou připojeny k **napěťovému žebříku** na vstupu **minus (-)**, který dělí **referenční napětí**, a na vstup **plus (+)** je přivedeno **měřené napětí**. Výsledky komparátorů jsou připojeny k **prioritnímu kodéru**, na jehož výstupu je výsledek převodu. Jeho výhodou je **rychlost** (převod probíhá v reálném čase kontinuálně pouze s nepatrným zpožděním), nevýhodou je cena a složitá realizace (velký počet **komparátorů**, nutnost mít velmi **přesné odpory**).
 - Pokud je Vin **poloviční** jako Vref, bude v **log. 1** spodní polovina výstupů z komparátorů.
 - Pokud je Vin **nulové**, nebude v **log. 1** žádný komparátor.
 - Pokud je Vin **rovné nebo větší** Vref, budou v **log. 1** všechny výstupy komparátorů.
- **Aproximační ADC**: Jedná se o **sekvenční** obvod, měřené napětí musí být **navzorkováno** a poté až probíhá převod. Funguje na principu binárního vyhledávání (**půlení intervalů**) správné hodnoty napětí, pracuje ale s lineární časovou složitostí $O(N)$, kde N je počet převáděných bitů (musí se zkontrolovat každý). Je tvořen: [[media/szz-05/media/image9.png]]

- **Sample and hold obvod**: navzorkuje napětí na začátku převodu a poté jej drží beze změny až do jeho konce,
- **DA převodník**: převádí aktuální **odhad** napětí z binární hodnoty na napětí,
- **Komparátor napětí**: porovnává **odhad** napětí naměřeného napětí s **měřeným** napětím,
- **SAR** (successive approximation register): obsahuje aktuální odhad naměřeného napětí v binární podobě. Na začátku je tato hodnota rovna polovině rozsahu (pouze **MSB** je v **log. 1**).

Převod probíhá tak, že **MSB** v **SAR** je nastaven na **jedna** a zbylé bity jsou **vynulovány**. Tato binární hodnota je převedena na napětí pomocí **DAC** a **komparátorem porovnána** s měřeným napětím. Pokud je odhad napětí **menší** než měřené napětí, zůstane **MSB** v **log. 1**, **jinak** je změněn na **log. 0**. V obou případech je nastaven druhý nejvíce signifikantní bit na **log. 1** a proces se opakuje. Teď už se ale po porovnání případně mění tento bit a na **log. 1** se nastavuje další.

[[media/szz-05/media/image3.png]]

DA převodník (digital-to-analog converter)

Převádí **digitální** signál (binární číslo) na **analogový** (spojitý). Většina DAC je tvořena **R-2R odporovým žebříkem** a případně ...

[[media/szz-05/media/image6.png]]

b0 je **LSB**, **b3** je **MSB**. Celkový odpor obvodu v místě **out** je **R**. Paralelně zapojené dva **2R** rezistory pod **b0**, mají odpor roven součtu jejich **převrácených** hodnot, tedy **R**, což znamená, že pod **b1**, vzniknou také paralelně zapojené dva **2R** ($2R$ a $2 \times 1R$ v sérii) rezistory atd. Napětí z **b0** bude po cestě k **out** rozděleno **čtyřikrát**, tedy doputuje tam $1/2^4 = 1/16$, z **b1** bude rozděleno **tříkrát** na **1/8**, z **b2** **dvakrát** na **1/4** a z **b3** jednou na **1/2** DAC Methods R2R Ladder. Na **out** takhle může být přivedeno napětí v rozmezí **0** až **Vcc**, které je na vstupech **b0-b3**, po šestnáctinových skocích. Pokud chceme převádět na jiná napětí, musí být **out** zapojeno ještě na **operační zesilovač**. How OpAmps Work - The Learning Circuit

[[media/szz-05/media/image15.png]]

Periferie

Jedná se o zařízení, která připojujeme k MCU, abychom například mohli zajistit interakci vestavěného systému s člověkem nebo řízení nějakého vnějšího obvodu.

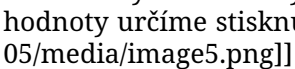
Displej

K MCU můžeme pomocí pinů připojit např. **8 segmentový displej**, viz obrázek (obecně je známý jako **7 segmentový**, na obrázku je ale i tečka). Čísla na displeji jsou poté zobrazována na základě nastavení **log 1**. a **log. 0** na výstupních portech MCU. V případě tohoto schématu **svítí segment**, když je odpovídající port MCU v **log. 0**, protože displej je připojen ke kladnému napětí. Při jiné implementaci to může být naopak.

[[media/szz-05/media/image10.png]]

Klávesnice

Připojení klávesnice k MCU lze řešit např.:

- Pomocí **děliče napětí** (rezistorový žebřík) jsou všechny tlačítka připojena k AD převodníku. Stisk různých kláves vyvolá **různě velká napětí**, která měříme a poté na základě naměřené hodnoty určíme stisknutou klávesu. (Na obrázku jsou spínače K1, K2, ... klávesy) 
- Další možností je použít kodér. Pomocí něj můžeme zakódovat hodnotu klávesy na binární číslo a to už přímo přivést na vstup MCU (na jednotlivé piny). Např. můžeme použít **16-4 prioritní kodér**, což nám umožňuje zapojit až **16 tlačítek** na **4 piny** MCU.

Bzučák/reproduktor, LED dioda, motor, ...

Všechny tyto periferie lze kontrolovat pomocí **pulzně šířkové modulace** (PWM, Pulse-width modulation), tj. rychlým střídáním log. 0 a log. 1 a regulováním **střídy** (duty cycle) na výstupu pinu MCU např. **pomocí časovače**. Druhou méně často ale přesnější možností je použít pro tyto účely **DA převodník**.







Odkazy:

- Seriové rozhraní: Sériová rozhraní u mikrokontrolerů – informatika

Zdroje

- SZZ okruh 5 — studijní materiály FIT BUT (szz-05.docx). Další obrázky: media/szz-05/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/04-hierarchie-pameti|4. Hierarchie paměti\]\]](#) · [další: \[\[okruhy/06-periferie-preruseni-dma-sbernice|6. Připojování periférií\]\]](#) [□](#)